

Chapitre I : Introduction à la Programmation par Contraintes

3 ING IA-GL

Belabed Amine

Université De Tlemcen – Faculté des sciences, Département d'Informatique

2025-2026

Définitions

Une Contrainte :

- Une contrainte est une **relation logique** (une propriété qui doit être vérifiée) entre différentes inconnues, appelées **variables**.

Définitions

Une Contrainte :

- Une contrainte est une **relation logique** (une propriété qui doit être vérifiée) entre différentes inconnues, appelées **variables**.
- Chaque variable prend ses valeurs dans un ensemble donné, appelé **domaine**. Une contrainte restreint les valeurs que peuvent prendre simultanément les variables.

Définitions

Une Contrainte :

- Une contrainte est une **relation logique** (une propriété qui doit être vérifiée) entre différentes inconnues, appelées **variables**.
- Chaque variable prend ses valeurs dans un ensemble donné, appelé **domaine**. Une contrainte restreint les valeurs que peuvent prendre simultanément les variables.
- **Exemple :**
 $X + Y = 1$; $(X = 0, Y = 1)$ et $(X = 1, Y = 0)$ si le domaine d'interprétation est $\mathbb{N}$.

Définitions

Type de contraintes :

- **Les contraintes numériques** : linéaires : $x * 3 + y * 4 < 5$; non linéaires : $x^2 = 2$, $\log(x) = y$.

Définitions

Type de contraintes :

- **Les contraintes numériques** : linéaires : $x * 3 + y * 4 < 5$; non linéaires : $x^2 = 2$, $\log(x) = y$.
- **Les contraintes booléennes** : $\neg a \vee b \Rightarrow c$, $\neg(a \vee b) \Leftrightarrow (c \wedge d)$.

Définitions

Type de contraintes :

- **Les contraintes numériques** : linéaires : $x * 3 + y * 4 < 5$; non linéaires : $x^2 = 2$, $\log(x) = y$.
- **Les contraintes booléennes** : $\neg a \vee b \Rightarrow c$, $\neg(a \vee b) \Leftrightarrow (c \wedge d)$.
- **Contrainte à domaine fini** : les valeurs que peuvent prendre les variables appartiennent à un domaine fini.

Utilisation de la programmation par contraintes

- Problèmes combinatoires (jeux, voyageur de Commerce, . . .) ;

Utilisation de la programmation par contraintes

- Problèmes combinatoires (jeux, voyageur de Commerce, . . .) ;
- Problèmes de planification (par exemple un emploi de temps, trafic ferroviare) ;

Utilisation de la programmation par contraintes

- Problèmes combinatoires (jeux, voyageur de Commerce,. . .) ;
- Problèmes de planification (par exemple un emploi de temps, trafic ferroviare) ;
- Problèmes d'ordonnancement (par exemple trouver une affectation de Tâches à exécuter a des machines) ;

Utilisation de la programmation par contraintes

- Problèmes combinatoires (jeux, voyageur de Commerce,. . .) ;
- Problèmes de planification (par exemple un emploi de temps, trafic ferroviare) ;
- Problèmes d'ordonnancement (par exemple trouver une affectation de Tâches à exécuter a des machines) ;
- Problèmes de placement (par exemple placer des objets dans un volume limité) ;

Utilisation de la programmation par contraintes

- Problèmes combinatoires (jeux, voyageur de Commerce, . . .) ;
- Problèmes de planification (par exemple un emploi de temps, trafic ferroviare) ;
- Problèmes d'ordonnancement (par exemple trouver une affectation de Tâches à exécuter a des machines) ;
- Problèmes de placement (par exemple placer des objets dans un volume limité) ;
- Problème d'optimisation : routage dans les réseaux.

Problèmes de satisfaction de contraintes (CSP)

Problèmes de satisfaction de contraintes : Définitions

Définition

Un CSP (Problème de Satisfaction de Contraintes) est un triplet (X, D, C) tel que :

- $X = \{X_1, X_2, \dots, X_n\}$: est l'ensemble des variables (les inconnues) du problème ;
 - D : fonction qui associe à chaque variable X_i son domaine $D(X_i)$;
 - $C = \{C_1, C_2, \dots, C_k\}$: est l'ensemble des contraintes.
-
- **Exemple** : $X = \{a, b, c, d\}$, $D(a) = D(b) = D(c) = D(d) = \{0, 1\}$,
 $C = \{a \neq b, c \neq d, a + c < b\}$.

Problèmes de satisfaction de contraintes : Définitions

- Une affectation (instanciation) A est un ensemble de couple variable/valeur :

$$A = \{X_1 \leftarrow v_1, X_2 \leftarrow v_2, \dots, X_n \leftarrow v_r\}$$

Tel que : $\forall X_i \in X, v_j \in D(X_i)$.

Problèmes de satisfaction de contraintes : Définitions

- Une affectation (instanciation) A est un ensemble de couple variable/valeur :

$$A = \{X_1 \leftarrow v_1, X_2 \leftarrow v_2, \dots, X_n \leftarrow v_r\}$$

Tel que : $\forall X_i \in X, v_j \in D(X_i)$.

- Une affectation est **partielle** si elle ne concerne qu'une partie des variables de X , et **totale** si elle concerne toutes les variables de X .

Problèmes de satisfaction de contraintes : Définitions

- Une affectation (instanciation) A est un ensemble de couple variable/valeur :

$$A = \{X_1 \leftarrow v_1, X_2 \leftarrow v_2, \dots, X_n \leftarrow v_r\}$$

Tel que : $\forall X_i \in X, v_j \in D(X_i)$.

- Une affectation est **partielle** si elle ne concerne qu'une partie des variables de X , et **totale** si elle concerne toutes les variables de X .
- Une affectation A est **valide (consistante)** par rapport à un ensemble de contraintes C si pour toute contrainte C_i de C , la relation définie par C_i est vraie pour les valeurs des variables définies dans A .

Problèmes de satisfaction de contraintes : Définitions

- Une solution (modèle) d'un CSP est une affectation totale et valide.

Problèmes de satisfaction de contraintes : Définitions

- Une solution (modèle) d'un CSP est une affectation totale et valide.
- Une affectation globalement consistante : une affectation partielle des variables de X et il est possible de la compléter en une solution du CSP.

Problèmes de satisfaction de contraintes : Définitions

- Une **solution (modèle)** d'un CSP est une affectation totale et valide.
- Une **affectation globalement consistante** : une affectation partielle des variables de X et il est possible de la compléter en une solution du CSP.

► **Exemple :**

$X = \{x, y, z\}$ et $D(x) = D(y) = D(z) = \{1, 2, 3\}$,

$C = \{x \neq y, y \neq z, x \neq z, y + z \leq 3\}$.

Problèmes de satisfaction de contraintes : Définitions

- Une solution (modèle) d'un CSP est une affectation totale et valide.
- Une affectation globalement consistante : une affectation partielle des variables de X et il est possible de la compléter en une solution du CSP.

► **Exemple :**

$X = \{x, y, z\}$ et $D(x) = D(y) = D(z) = \{1, 2, 3\}$,

$C = \{x \neq y, y \neq z, x \neq z, y + z \leq 3\}$.

★ $x=3, y=1$ est globalement consistante ($z=2$),

Problèmes de satisfaction de contraintes : Définitions

- Une solution (modèle) d'un CSP est une affectation totale et valide.
- Une affectation globalement consistante : une affectation partielle des variables de X et il est possible de la compléter en une solution du CSP.

► **Exemple :**

$X = \{x, y, z\}$ et $D(x) = D(y) = D(z) = \{1, 2, 3\}$,

$C = \{x \neq y, y \neq z, x \neq z, y + z \leq 3\}$.

★ $x=3, y=1$ est globalement consistante ($z=2$),

★ $x=1, y=3$ n'est pas globalement consistante.

Problèmes de satisfaction de contraintes

Types de CSP

- **CSPs binaires** : si toutes les contraintes du CSP sont binaire.
- **CSPs n-aires** : s'il existe des contraintes qui définissent des relations entre plusieurs variables (> 2).

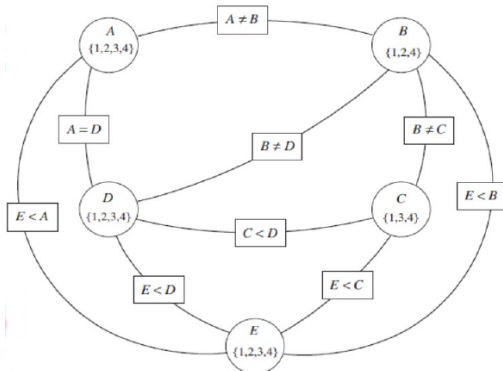
Remarque

Tout CSP peut être représenté comme un CSP binaire.

Problèmes de satisfaction de contraintes

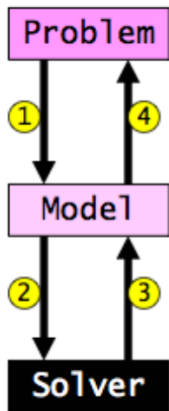
Le réseau de contraintes

- Un CSP binaire peut être représenté comme un réseau (graphe) de contraintes ;
- Exemple :



Problèmes de satisfaction de contraintes : utilisation

Pour résoudre un problème à l'aide de CSP on passe par les étapes décrites par la figure suivantes :



Problèmes de satisfaction de contraintes : Exemples

- Puzzles Crypto-Arithmétique

$$\begin{array}{r} \text{SEND} \\ + \text{MORE} \\ \hline \text{MONEY} \end{array}$$

$$\begin{array}{r} 9567 \\ +1085 \\ \hline 10652 \end{array}$$

- CSP ?

Problèmes de satisfaction de contraintes : Exemples

- Puzzles Crypto-Arithmétique

$$\begin{array}{r} \text{SEND} \\ + \text{MORE} \\ \hline \text{MONEY} \end{array}$$
$$\begin{array}{r} 9567 \\ + 1085 \\ \hline 10652 \end{array}$$

- CSP ?

variables: D, E, M, N, O, R, S, Y

domains: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

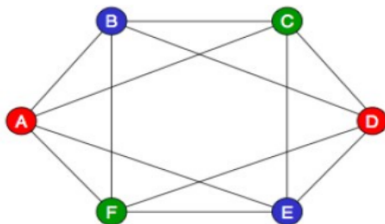
constraints: $D \neq 0$, $M \neq 0$,

$$\begin{aligned} &1000 \cdot S + 100 \cdot E + 10 \cdot N + D + \\ &1000 \cdot M + 100 \cdot O + 10 \cdot R + E = \\ &10000 \cdot M + 1000 \cdot O + 100 \cdot N + 10 \cdot E + Y \end{aligned}$$

$D \neq E, D \neq M, D \neq N, D \neq O, D \neq R, D \neq S, D \neq Y, E \neq M, E \neq N, E \neq O, E \neq R,$
 $E \neq S, E \neq Y, M \neq N, M \neq O, M \neq R, M \neq S, M \neq Y, N \neq O, N \neq R, N \neq S, N \neq Y,$
 $O \neq R, O \neq S, O \neq Y, R \neq S, R \neq Y, S \neq Y$

Problèmes de satisfaction de contraintes : Exemples

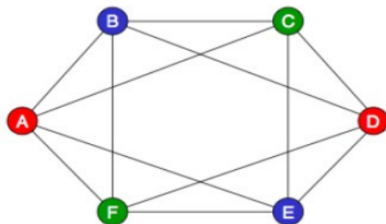
- Coloriage de graphe



- CSP ?

Problèmes de satisfaction de contraintes : Exemples

- Coloriage de graphe



- CSP ?

Variables : A, B, C, D, E, F

Domaines : {rouge, bleu, vert, jaune, blueciel, violet}

Contraintes : $A \neq B$, $A \neq C$, $A \neq E$, $A \neq F$, $B \neq C$, $B \neq D$, $B \neq F$,
 $C \neq D$, $C \neq E$, $E \neq E$, $D \neq F$, $E \neq F$

Problèmes de satisfaction de contraintes : Exemples

- **Le coffre-fort** : Une personne a oublié la combinaison pour ouvrir son coffre-fort. La combinaison du coffre-fort comporte 9 chiffres au total. Elle se souvient que :
 - ① Tous les chiffres sont différents ;
 - ② Tous les chiffres sont dans la plage 1..9 ;
 - ③ La différence entre le 4e et le 6e chiffre est égale au 7e chiffre ;
 - ④ Le 1er chiffre n'est pas 1, le 2ème chiffre n'est pas 2, et ainsi de suite .
 - ⑤ Le produit des trois premiers chiffres est égal à la somme des deux derniers chiffres.
 - ⑥ La somme des 2ème, 3ème et 6ème chiffres est inférieure au 8ème chiffre,
 - ⑦ Le dernier chiffre est également inférieur au 8ème chiffre.
- CSP ?

Problèmes de satisfaction de contraintes : Exemples

- Le coffre-fort :

- ▶ Les variables : $X = \{X1, X2, \dots, X9\}$.
- ▶ Les domaines : $D(X) = \{1, 2, \dots, 9\}$.
- ▶ Les contraintes :
 - ★ $AllDifferent(X1, X2, \dots, X9)$;
 - ★ $\forall i \in \{1, \dots, 9\} : X_i \neq i$;
 - ★ $X4 - X6 = X7$;
 - ★ $X1 \times X2 \times X3 = X8 + X9$;
 - ★ $X1 + X2 + X6 < X8$;
 - ★ $X9 < X8$.

Problèmes de satisfaction de contraintes : Exemples

- Le carré magique :

4	9	2
3	5	7
8	1	6

- CSP ?

Problèmes de satisfaction de contraintes : Exemples

- Le carré magique :

- ▶ Les variables : $X = \{X_{ij} / ij \in [1..n]\}$.
- ▶ Les domaines : $D(X) = \{1, 2, \dots, n^2\}$.
- ▶ Les contraintes : On suppose que la somme magique égale à S
 $(n \times (n^2 + 1)/2)$
 - ★ $AllDifferent(X_{11}, X_{12}, \dots, X_{nn})$;
 - ★ Les lignes : $\forall i \in [1..n], \sum_{j=1}^n X_{ij} = S$;
 - ★ Les colonnes : $\forall j \in [1..n], \sum_{i=1}^n X_{ij} = S$;
 - ★ diagonale 1 : $\forall i, j \in [1..n], Si i = j \sum X_{ij} = S$;
 - ★ diagonale 1 : $\forall i, j \in [1..n], Si i + j = n + 1 \sum X_{ij} = S$;

Problèmes de satisfaction de contraintes : Exemples

- n-Reines (4-reines, 8-reines, ...)

	Q		
			Q
Q			
		Q	

- CSP ?

Problèmes de satisfaction de contraintes : Exemples

- n-Reines

	Q		
			Q
Q			
		Q	

- CSP ? (4-reines)

- ▶ Variables : $X = \{X_1, X_2, X_3, X_4\}$.
- ▶ Domaines : $D(X_1) = D(X_2) = D(X_3) = D(X_4) = \{1, 2, 3, 4\}$.
- ▶ Les reines doivent être sur des lignes différentes :
 $Clig = \{X_i \neq X_j | i, j \in \{1, 2, 3, 4\}, i \neq j\}$.
- ▶ Les reines doivent être sur des diagonales montantes différentes :
 $Cdm = \{X_i + i \neq X_j + j | i, j \in \{1, 2, 3, 4\}, i \neq j\}$.
- ▶ Les reines doivent être sur des diagonales descendantes différentes :
 $Cdd = \{X_i - i \neq X_j - j | i, j \in \{1, 2, 3, 4\}, i \neq j\}$.

- Les contraintes de diagonales peuvent s'écrire :

$$|X_i - X_j| \neq j - i, \forall i, j \in \{1, 2, 3, 4\}, i \neq j$$

Problèmes de satisfaction de contraintes : Exemples

- Le Sudoku :

	3		4		5		7	
6	2			8		4		
7					1			9
2		6			3	8		
						2		3
	1	3	6			9	5	
		8		4	7			
								6
		9		5		3	8	2

- CSP ?

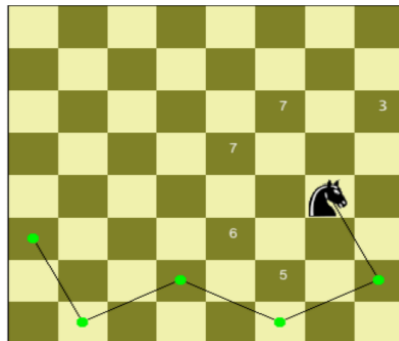
Problèmes de satisfaction de contraintes : Exemples

● Le Sudoku :

- ▶ Variables : X_{ij} : le chiffre dans le carre (i,j) avec $i, j \in [1, 9]$.
- ▶ Domaines : $D(X_{ij}) = \{1, \dots, 9\}, \forall (i, j) \in [1, 9]$.
- ▶ Contraintes :
 - ★ Les chiffres dans chaque ligne sont différents :
 $\text{AllDifferent}(X_{i1}, X_{i2}, \dots, X_{i9}), 1 \leq i \leq 9;$
 - ★ Les chiffres dans chaque colonne sont différents :
 $\text{AllDifferent}(X_{1j}, X_{2j}, \dots, X_{9j}), 1 \leq j \leq 9;$
 - ★ Les chiffres dans chaque bloc 3×3 sont différents :
 $\text{AllDifferent}(X_{3k+1,3l+1}, X_{3k+1,3l+2}, \dots, X_{3k+3,3l+3}), 0 \leq k, l \leq 2.$
 - ★ Les cases remplies :
 $X_{12} = 3, X_{21} = 6, X_{22} = 2, X_{14} = 4, \text{etc.}$

Problèmes de satisfaction de contraintes : Exemples

- Le cavalier d'Euler :



- CSP ?

Problèmes de satisfaction de contraintes : Exemples

Le cavalier d'Euler :

- Variables et domaines :

- ▶ X_i et Y_i , avec $i = 0, \dots, 64$, pour désigner les coordonnées (X_i, Y_i) de la case où se trouve le cavalier après le **i-ème** pas.
- ▶ $D(X_i) = D(Y_i) = \{0, 1, \dots, 7\}$
- ▶ n_i , avec $i = 0, \dots, 64$, pour désigner le numéro de la case où se trouve le cavalier après le i-ème pas.
- ▶ $D(n_i) = \{0, 1, \dots, 63\}$

- Les contraintes :

- ▶ $n_i = 8 * X_i + Y_i$
- ▶ Le cavalier ne peut pas repasser sur une même case (sauf pour la case de départ) :
 $AllDifferent(n_1, n_2, \dots, n_{64})$.
- ▶ Le cavalier doit revenir à son point de départ :
 $n_0 = n_{64}$

Problèmes de satisfaction de contraintes : Exemples

Le cavalier d'Euler :

- Les contraintes :
 - ▶ Les contraintes de déplacement du cavalier (en L) :

$$\forall i \in \{0, \dots, 63\} \left\{ \begin{array}{l} |X_i - X_{i+1}| + |Y_i - Y_{i+1}| = 3 \\ X_i \neq X_{i+1} \\ Y_i \neq Y_{i+1} \end{array} \right.$$

CSP : Algorithmes de résolution

L'algorithme génère et teste

- **Principe** : énumérer toutes les affectations totales possibles jusqu'à en trouver une qui satisfasse toutes les contraintes.

- **Algorithme** :

Tant que toutes les contraintes ne sont pas satisfaites :

- ① Nouvelle instanciation de toutes les variables ;
- ② Évaluation des contraintes.

- **Exemple** : soit le $CSP(V, D, C)$ suivant :

- ▶ $V = \{X, Y, Z\}$;
- ▶ $D(X) = D(Y) = D(Z) = \{1, 2\}$;
- ▶ $C = \{X = Y, X \neq Z, Y > Z\}$.

CSP : Algorithmes de résolution

L'algorithme génère et teste

- **Solution :**

X	Y	Z	Contraintes
1	1	1	F
1	1	2	F
1	2	1	F
1	2	2	F
2	1	1	F
2	1	2	F
2	2	1	V
2	2	2	■

CSP : Algorithmes de résolution

L'algorithme simple backtracking

- **Algorithme :**

Tant que toutes les variables ne sont pas instanciées :

- 1 Instanciation d'une nouvelle variable ;
- 2 Évaluation des contraintes totalement instanciées :

Si le système courant est inconsistent $\rightarrow$ **retour arrière** (Choix d'une nouvelle valeur pour la dernière variable instanciée pour laquelle il reste des valeurs disponibles)

- **Exemple :** soit le $CSP(V, D, C)$ suivant :

- ▶ $V = \{X, Y, Z\}$;
- ▶ $D(X) = D(Y) = D(Z) = \{1, 2\}$;
- ▶ $C = \{X = Y, X \neq Z, Y > Z\}$.

CSP : Algorithmes de résolution

L'algorithme simple backtracking

- **Solution :**

X	Y	Z	Contraintes
1	1	1	F
-	-	2	F
-	2		F
2	1		F
	2	1	V

CSP : Algorithmes de résolution

L'algorithme d'anticipation par Vérification en Avant (forward checking)

- **Algorithme :**

- ① On affect successivement des valeurs aux variables ;
- ② A chaque fois qu'on affecte une valeur à une nouvelle variable X_i , on supprime du domaine des variables non affectées et en relation avec X_i les valeurs qui ne sont pas compatibles avec cette affectation.

Exemple : soit le $CSP(V, D, C)$ suivant :

- ▶ $V = \{X, Y, Z\}$;
- ▶ $D(X) = D(Y) = D(Z) = \{1, 2, 3, 4\}$;
- ▶ $C = \{X = Y, X \neq Z, Y > Z\}$.

CSP : Algorithmes de résolution

L'algorithme d'anticipation par Vérification en Avant

- **Solution :**

affectaion	D(X)	D(Y)	D(Z)	Contraintes
	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}	
X=1	1	{1}	{2, 3, 4}	
Y=1	1	1	{}	backtrack
X=2	2	{2}	{1, 3, 4}	
Y=2	2	2	{1}	
Z=1	2	2	1	V

Consistance dans les CSPs

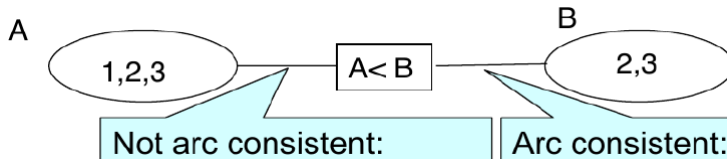
Consistance de nœud (de variable)

- Un nœud (variable) X_i est consistant si toute valeur v dans $D(X_i)$ satisfait toutes les contraintes unaires relatives à X_i .
- Un $CSP(X, D, C)$ est consistant de nœud si pour toute variable X_i de X , et pour toute valeur v de D_i , l'affectation partielle (X_i, v) satisfait toutes les contraintes unaires de C .

Consistance dans les CSPs

Consistance d'arc

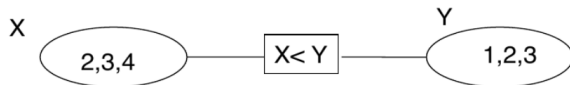
- Un arc $(X_i, C(X_i, X_j))$ est consistant, si pour toute valeur v_i de $D(X_i)$, il existe une valeur v_j de $D(X_j)$ telle que la contrainte $C(X_i = v_i, X_j = v_j)$ est valide.
- Un $CSP(X, D, C)$ est consistant d'arc si tout couple de variables (X_i, X_j) de X , et pour toute valeur v_i de $D(X_i)$, il existe une valeur v_j appartenant $D(X_j)$ telle que l'affectation partielle $(X_i, v_i), (X_j, v_j)$ satisfasse toutes les contraintes binaires de C .
- Exemple :



Consistance dans les CSPs

Rendre un arc consistant :

- Si un arc $(X_i, C(X_i, X_j))$ n'est pas consistant :
 - ▶ supprimer toutes les valeurs v_i de $D(X_i)$ dont il n'existe pas un support v_j de $D(X_j)$.
 - ▶ Cette suppression rend cet arc consistant et elle n'affecte jamais les solutions du CSP.
- Exemple : rendre l'exemple suivant arc consistant



Consistance dans les CSPs

Rendre un CSP arc consistant : Algorithme AC1

- **Principe** : répéter la suppression des valeurs jusqu'à la stabilisation (pas de suppression possible).
- **Algorithme** :

répéter

terminé $\leftarrow$ VRAI;

pour chaque *contrainte* $C(x, y)$ **faire**

si *il existe dans* D_x *des valeurs qui n'ont pas de support dans* D_y **alors**

 les supprimer;

terminé $\leftarrow$ FAUX;

jusqu'à *terminé* = VRAI;

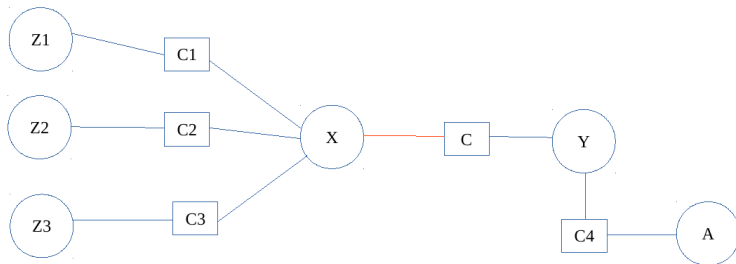
Consistance dans les CSPs

Rendre un CSP arc consistant : Algorithme AC3

● Principe :

- ▶ Considérer à tour de rôle chaque arc de notre réseau de contrainte et le rendre consistant.
- ▶ Reconsidérer les arcs qui sont susceptibles de devenir inconsistants. Un arc $(X_i, C(X_i, X_j))$ est reconsidéré si le domaine de X_j est réduit.

● Exemple :



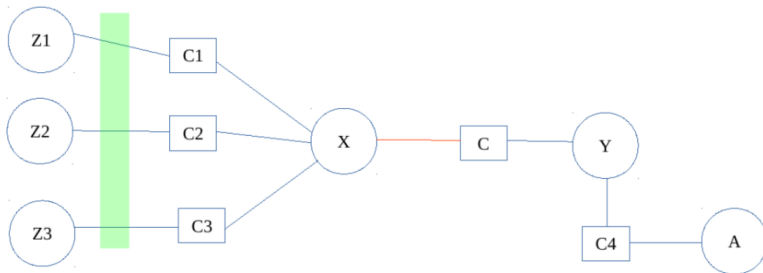
Consistance dans les CSPs

Rendre un CSP arc consistant : Algorithme AC3

● Principe :

- ▶ Considérer à tour de rôle chaque arc de notre réseau de contrainte et le rendre consistant.
- ▶ Reconsidérer les arcs qui sont susceptibles de devenir inconsistant. Un arc $(X_i, C(X_i, X_j))$ est reconsidéré si le domaine de X_j est réduit.

● Exemple :



Consistance dans les CSPs

Rendre un CSP arc consistant : Algorithme AC3

- **Algorithme :**

$\text{àTester} \leftarrow \{C(x, y)\}_{C(x, y) \in C};$

pour chaque $C(x, y) \in \text{àTester}$ **faire**

$\text{àTester} \leftarrow \text{àTester} \setminus \{C(x, y)\};$

 supprimer les valeurs de D_x qui n'ont pas de support dans D_y ;

si au moins une valeur a été supprimée **alors**

$\text{àTester} \leftarrow \text{àTester} \cup \{C(z, x) : \exists C(z, x) \in C, z \neq x\};$

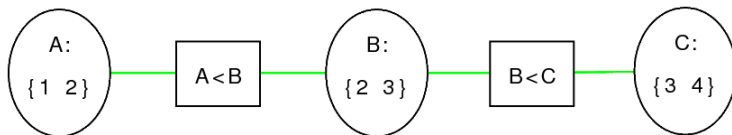
- **Exemple :** En utilisant l'algorithme AC3 rendre le CSP suivant arc consistant :



Consistance dans les CSPs

Rendre un CSP arc consistant : Algorithme AC3

- **Exemple** : Solution



Consistance dans les CSPs

L'algorithme de consistance d'arc, produit 3 cas :

- ❶ Chaque domaine a une valeur unique (c'est une solution du CSP) ;
- ❷ Au moins un domaine est vide : pas de solution ;
- ❸ Au moins un domaine a plusieurs valeurs : il existe une ou plusieurs solutions, ou pas de solution.

Recherche par division de domaine

Après l'application de l'algorithme de consistance d'arcs, s'il existe des domaines avec plusieurs valeurs, on peut :

- 1 Utiliser un algorithme de recherche ex : backtracking (dfs) ;
- 2 Diviser le CSP en un nombre disjoint de sous CSPs : la solution globale est l'union de toutes les solutions.

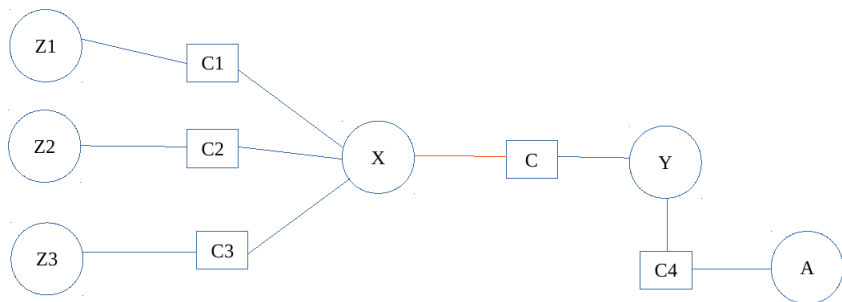
Exemple : $CSP(X, D, C)$, $X = \{A, B, C\}$, $D(X) = \{1, 2\}$,
 $C = \{A \neq B, A \neq C, B \neq C\}$.

- $CSP1(X, D, C)$, $X = \{A, B, C\}$, $D(B, C) = \{1, 2\}$, $D(A) = \{1\}$,
 $C = \{A \neq B, A \neq C, B \neq C\}$.
- $CSP2(X, D, C)$, $X = \{A, B, C\}$, $D(B, C) = \{1, 2\}$, $D(A) = \{2\}$,
 $C = \{A \neq B, A \neq C, B \neq C\}$.

Recherche par division de domaine

Les arcs a reconsidérés après la division de domaine :

- **Exemple** : si on divise le domaine du variable X quels sont les arcs a reconsidérés ?



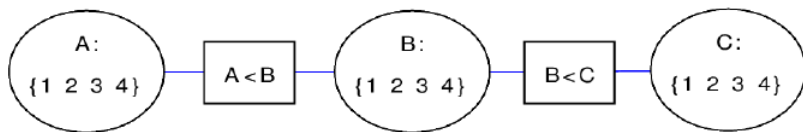
Recherche par division de domaine

Algorithme de consistance d'ars avec division de domaine

```
Procédure AC_DS(CSP(X,D,C),Larcs )
output : l'ensemble des modèles du CSP
DOM←AC(CSP(X,D,C))
if DOM contient un domaine vide alors retourner {} ;
if quelque soit Xi in X, DOM(Xi) est un singleton alors
    retourner {X1=v1,...,Xn=vn} ;
choisir une variable Xi de X ;
[D1, ...Dn]← partitionnez DOM(Xi) a n domaine non vide ;
models ← {}
for i=1 à n do
    DOMi←DOM avec DOM(Xi) remplacé par Di ;
    Larcs←{(Z,c) | Z in X\{Xi} et c inclus Xi et c inclus Z} ;
    models ← models U AC_DS(CSP(X,DOMi,C),Larcs ) // réursive
return models ;
end.
```

Recherche par division de domaine

Exemple :



Utilisation des heuristiques

Pour améliorer l'efficacité de la recherche par backtrack

- heuristiques qui se base sur l'ordre des variables :
- heuristiques qui se base sur l'ordre à base d'ordre des valeurs.

Utilisation des heuristiques

Heuristiques qui se base sur l'ordre des variables

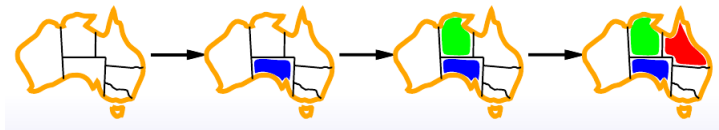
- Heuristique des **valeurs minimum restantes (MVR)**
 - ▶ choisir la variable avec le moins de valeurs compatibles restantes :
- **Exemple :**



Utilisation des heuristiques

Heuristiques qui se base sur l'ordre des variables

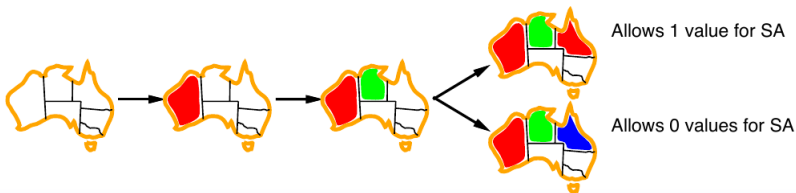
- Heuristique du **degré** (degree heuristic)
 - ▶ Si plusieurs variables ne peuvent pas être départagées par l'heuristique MVR ;
 - ▶ Choisir la variable qui a le plus de contraintes à respecter parmi les variables restantes.
- **Exemple :**



Utilisation des heuristiques

heuristiques qui se base sur l'ordre à base d'ordre des valeurs ;

- Pour une variable donnée, choisir une valeur qui invalide le moins de valeurs possibles pour les variables non encore assignées.
- **Exemple :**



- Artificial Intelligence : A Modern Approach, by Stuart Russell and Peter Norvig (Third Edition), chapitre 5.
- Thèse, Arnaud Malapert, "Techniques d'ordonnancement d'atelier et de fournées basées sur la programmation par contraintes", chapitre 2, [http ://www.i3s.unice.fr/ malapert/thesis/split/chapitre2.pdf](http://www.i3s.unice.fr/malapert/thesis/split/chapitre2.pdf).
- [http ://www.aispace.org](http://www.aispace.org).
- Cours, Programmation par contraintes, Odile PAPINI, ESIL Université de la méditerranée.